Which movie might you like?

Vlad Mutilica

May 2022

1 Declaration

This work has not been previously accepted in substance for any degree and is not being con- currently submitted in candidature for any degree.

Signed ..... (candidate)

Date:

2 Statement 1

This thesis is the result of my own investigations, except where otherwise stated. Other sources are acknowledged by footnotes giving explicit references. A bibliography is appended.

Signed ..... (candidate)

Date:

3 Statement 2

I hereby give my consent for my thesis, if accepted, to be made available for photocopying and inter library loan, and for the title and summary to be made available to outside organisations.

Signed ..... (candidate)

Date:

Abstract

Nowadays people are facing an increasing number of choices in different aspects of their lives. For example, in media, namely movies, books and music, as well as in other taste-based areas such as restaurants and food or holiday destinations, but, most importantly, in sectors such as health insurance, job searches and education.

This increased variety of options poses an issue for human beings as we are naturally bad at making decisions, especially when confronted with many options. People get quickly overwhelmed and end up making sub optimal choices or not choosing at all - hence the term "analysis paralysis".

This is where technology can cover for our weaknesses. To be more precise: machine learning. When presented with an issue such as the one described, the computer's capacity to go through a vast array of options in a very small amount of time and store and compare the benefits of each choice relieves the user from the difficulty of making a decision by fetching the most suitable alternatives and presenting them.

4 Acknowledgements

I would like to thank my supervisor, Alma Rahat, for his support throughout this journey and also his insightful information he gifted me. And I would like to thank my family and friends for their support during my degree.

My appreciation also goes to Chinmay Rane and Yohan Jeong for inspiring this project's implementation.

Contents

5 Introduction

5.1 Motivation

The idea of using machine learning in decision making has been the status quo for many mainstream media streaming platforms of today mainly because it increases the user's satisfaction. One other benefit of machine learning recommended systems is that the users are being led to other genres that they would not normally choose by themselves, which consequently spreads their experience to a larger area of genres and outlets. This, in turn, benefits the smaller content outlets as well.

The main two entities that can be found in any Recommender System are the users (customers) and the items (products). The user is any person that utilizes the Recommender System by first providing his opinion about some items and afterwards being sent back recommendations about other items available in the system's backlog. The input to a Recommender System depends on the type of employed filtering algorithm. Usually, the input can be classified in one of the following categories:

Ratings store the user's feedback on items. They usually follow a numerical scale – for example 1-5 – and are taken from the users. Another commonly used rating scheme is the binary one, modelled by the like-dislike feature present on many platforms today. Ratings can also be obtained implicitly from the cookies provided by the user. Some examples would be age, gender and interests, characteristics from which precise conclusions can be drawn.

Demographic data refers to data regarding the user, some examples being: gender, age, education, location and so on. One characteristic of this type of data is a high gathering difficulty.

Content data relies on a textual analysis of the related items to the ones provided by the user. The features gathered by this analysis can be used together with filtering algorithms to conclude a user profile.

The goal of any Recommender System is to provide suggestions about new items or to predict the utility of a new item for a specific user. This process relays on the provided input, which is usually concluded from previously recorded preferences of the given user.

5.2 Aims & objectives

The aims of this project are to study and develop a transparent recommendation system that can also ease the cold starting problem. To achieve this goal we considered that Content-based filtering and User-based filtering were the best options for the chosen task, since they are deterministic algorithms and they offer a clear view of how the suggestions are being provided. When both methods were in place and we could get a clear grasp on the results, we realized that this project needed a way to select and iterate through the recommendations logically, and we therefore decided to sort these results in multiple Pareto sets. Furthermore, the recommendation system makes use of a deep learning model to

predict the recommended movie's rating of the target user based on the rating history of all the users.

5.3 Structure

This paper will be divided into three sections:

5.3.1 Section 1: Background

This section will explore and showcase other examples of machine learning based recommendation systems used by media sharing companies nowadays. This section will focus on the following companies:

- YouTube
- Netflix
- Spotify

5.3.2 Section 2: Approach & implementation

The following section will contain all the steps required to make the addressed movie recommendation algorithm starting from gathering the data sets all the way to the final results. Subsequently, it will also provide a brief overview of the tools used in the development process.

5.3.3 Section 3: Project management

In this last section we will have a reflection on some key aspects of the development of the project.

- The first subsection will include a timeline reflection in which we will discuss how the project has developed and compare the results and process to the initial plan.
- In the second subsection the initial direction and the actual outcome of the project will be discussed and reasoned.
- The third section will address the encountered difficulties and provide their mitigation.

5.3.4 Section 3: Conclusion

This section is divided into three subsections:

- Lessons Learned: This section includes the main conclusions drawn from the development of the project.
- Future work: This section includes some ideas that could improve the performance of this project.

- Summary: This section concludes the project development and outlines the key aspects of this paper.

6 Background

6.1 Related work

Recommendation Systems represent a perfect example of the mainstream applicability of large scale data processing. Applications search, Internet videos and music make use of similar approaches to mine large volumes of data to cater to their users' needs in a personalized manner.

From all the other sources, we found that content-based collaborative filtering and user-based collaborative filtering seem to represent two very viable algorithms in sorting the items by relevance. The way in which these work is by ranking all the movies by a score computed based on content similarity (genre similarity in our case) or by tracing similar users and recommending movies from one to another. To this process we also added a way of categorizing the items in different shells by using multi objective optimization. This addition not only provides the best of both algorithms, but also provides the user with the option to cycle through the recommendations and further mitigates the problem of predicting something as subjective as preference.

6.1.1 YouTube

YouTube is one of the most popular content sharing platforms, holding millions of users around the world. Besides the astonishing amount of viewers YouTube hosts, it also has the largest video archive that is growing at an expeditious rate as 1 hour of content is being uploaded to YouTube every second. Even though this quantity of data can draw in users of highly varied background and different interests, it also comes with its drawbacks. Analyzing this enormous influx of data constitutes an arduous task.

The standard YouTube's recommendation system infrastructure relies on content based recommendation and collaborative recommendation. Another feature of this system is a hybrid recommendation approach, that consists of both techniques mentioned earlier and which is designed through a deep learning model. This model plays a crucial role as it keeps the addressed recommendations focused on the user's interests. Users are also provided with a search field and with the ability to leave their feedback on the videos through comments, likes and dislikes. One crucial metric in YouTube's recommendation system is the video watch time.

These previously mentioned recommendation systems are taking into account not only the content history of the target user but also the videos that had a good reach for the users with similar interests to the target user. The collaborative recommendation systems not only compares the nature of the content the users interact with but also the way they engage with the video. People with

similar profiles are virtually linked to one another and are being recommended videos from each others' histories.

Regarding content base recommendation, the user is being offered suggestions based on the kind of content they have previously liked. Some parameters taken into account are the title and the description of the video. They can be textually analyzed to conclude the genre and context of the video from which the similarity to the user is being computed. The videos with the highest similarity are being recommended since they are considered a good fit for the user. [1]

6.1.2 Netflix

Netflix is an American film streaming and original production company offering subscription-based video on demand. As of October 19, 2021, Netflix had 214 million subscribers. The motivation behind the service is that their subscribers need help in choosing what to watch since people not naturally prone to decision making if they are presented with many options. Netflix blends personalised PVR (personalised video ranker) with an unpersonalised series of suggestions. [4]

Regarding the ranking function that Netflix uses to optimise utilization, item popularity would be an obvious baseline since usually a user is prone to watch what all the other subscribers are watching. However, popularity is the opposite of personalization, producing the same ordering of items for every member. Thus, finding an equilibrium between popularity and personalization becomes the goal. Hence making satisfying members with varying tastes possible. One approach would be to using the predicted rating of each member for each item as an adjunct to item popularity. With these features a ranking prediction model can be constructed.

One of the key aspects in creating an optimal recommender system is represented by the data. The volume and the quality of the data owned by Netflix opens a myriad of opportunities. Netflix holds a data set consisting of billions of user ratings and constantly receive millions of ratings daily. The items in the data set Netflix holds is also very rich in metadata, operating with issues as genres, directors, actors, release dates, revenue and so on. One of the later included features to the data set is social data.

Regarding the models Netflix uses to tackle the given task of providing users with movie recommendations, a vast array of machine learning approaches is used, from unsupervised learning methods such as clustering – to supervised classifiers proven to be a good fit in multiple contexts. Some of the disclosed methods used include: Elastic nets, Logistic regression, Singular Value Decomposition (SVD), Latent Dirichlet Allocation, Matrix factorization, Gradient boosted decision trees and Affinity propagation.

In order to design Big Data solutions it is not only the large and feature-rich data sets that are required, but also a smart design in algorithm implementation. One of the important aspects of a good model is the scalability and the re-activeness of the model to new incoming data. In order to satisfy these

requirements Netflix approaches near-line computation. Even though there are multiple solutions to scaling offline computations – one example being Hadoop, a collection of open source utilities that aids large data computation by using a computer network and Map Reduce model. Offline computation implies no limitations on data volume nor complexity, as it runs in a batch manner. Despite that offline computation offers many advantages, it cannot solve latency related problems and cannot achieve re-activeness to user events. However, offline results can easily become irrelevant over a short period of time since they miss the new data input. To aid all the previously mentioned drawbacks Netflix has also resorted to online computation as it offers a quick response to user events and the incoming data stream. The drawback of this approach is the difficulty of fitting complex models.

Near-line computation has been the chosen design due to the fact that it compromises between offline and online computation. In this case the computation performs in an online manner. However, the recommendations are not directly sent back but stored. The computations are being triggered by user events, which make it more responsive between requests. [2]

6.1.3 Spotify

Another media streaming platform that involves machine learning in providing the users with their desired piece of content is Spotify, a Swedish media services provider founded on 23 April 2008, which holds over 365 million monthly active users, including 165 million paying subscribers offering access to over 70 million music tracks and 2.9 million podcast titles (June 30 2021). [6]

For any content streaming service that are offering on-demand features, there are two main categories into which the streamed content can be divided: algorithm-driven ("programmed") listening and user-driven ("organic") listening. On Spotify, the subscribers can look up songs, and create and share playlists. These functionalities are classified as organic listening as streams are driven by user action. Alternately, the users can listen to an algorithmically personalised playlist (Discover weekly), radio stations or curated playlists algorithmically generated by a "seed" song, as the whole playlist revolves around the characteristics of the initial song i.e genre and related artists. Even though the stream has been initiated by the user, songs are chosen without the necessity of any feedback. These streams are classified as programmed.

One of the goals of Spotify's music recommendation system is to measure musical diversity at a global scale and analyze its link with the user and the platform outcomes. Their recommendation system stores the information by using embeddings that encode the latent factors between the content and the users. This storing method maintains the link between items that are similar to one another. This is achieved by projecting the related items close to each other in the embedding space.

The embedding space is being produced with the use of the world2vec, algorithm which is being trained on the playlists generated by the users. This algorithm is mainly used in natural language processing to embed words from a

document. In the given context of music, the playlists are treated as documents and the songs as terms inside the documents. This implementation makes use of the bags-of-words model. The task is to predict the songs in the middle of each context. 850 million playlists were used to build the embedding space. These playlists have been filtered based on the meaningfulness they provide. Some of the factors that were used to decide the quality of the playlists were the length and also the frequency the user interacts with that playlist. The embedding space is used to define a user's preference profile as the average of the embeddings they have interacted with within a time frame. The vectors used are 40 dimensional and are being retrained daily to intake the new content uploaded to the platform. The vector similarity is calculated by applying cosine similarity to the distance between the vectors. [3]

7 Implementation & achievements

7.1 Implementation

7.1.1 Gathering data sets

As this was my first Machine Learning project, gathering data was a fairly arduous challenge. Firstly we resorted to the IMDB data set that provided a very thorough description of the movies provided, but nonetheless lacked consistency and user movie rating history. After researching the data set we realised that it misses some features that we would need for our approach. After some more investigation, we came across another data set widely used in movie recommendation systems called Movie Lens, which provides a large number of users with movie ratings. This data set format came in different sizes but for the scale of this project Movie lens 2 million seemed to be the best suit. Finally, after getting hold of both data sets we managed to link them together and take advantage of both of them.

7.1.2 Collaborative filtering

Collaborative filtering systems compute a user's preference for items based on their input. This process is achieved by sharing scores between people with similar preferences and connecting that user's interests with the recorded preferences of a set of users or by calculating the key features of the items they appreciate and suggest items that have suiting features. [5]

7.1.3 Content Based Collaborative filtering

In this project Content based collaborative filtering was used to compute genre similarity between the set of movies that the user likes and the movies that the data-sets have to offer. This process has been achieved by using Term frequency–inverse document frequency (tf-idf) for quantifying the movie genres and cosine similarity to compute the similarity between the movies.

```
[(0, 'action'), (1, 'adventure'), (2, 'animation'), (3, 'children'), (4, 'comedy'), (5, 'crime'),
(6, 'documentary'), (7, 'drama'), (8, 'fantasy'), (9, 'horror'), (10, 'imax'), (11, 'musical'),
(12, 'mystery'), (13, 'noir'), (14, 'romance'), (15, 'scifi'), (16, 'thriller'), (17, 'war'), (18,
'western')]
```

Figure 1: Movie genre encoding.

	0	1	2	3	4	5	6	7	8	9	10
0	0.000000	0.419059	0.518415	0.505858	0.263711	0.000000	0.0	0.000000	0.479790	0.000000	0.0
1	0.000000	0.515162	0.000000	0.621867	0.000000	0.000000	0.0	0.000000	0.589821	0.000000	0.0
2	0.000000	0.000000	0.000000	0.000000	0.602191	0.000000	0.0	0.000000	0.000000	0.000000	0.0
3	0.000000	0.000000	0.000000	0.000000	0.544364	0.000000	0.0	0.427589	0.000000	0.000000	0.0
4	0.000000	0.000000	0.000000	0.000000	1.000000	0.000000	0.0	0.000000	0.000000	0.000000	0.0
5	0.576139	0.000000	0.000000	0.000000	0.000000	0.610335	0.0	0.000000	0.000000	0.000000	0.0
6	0.000000	0.000000	0.000000	0.000000	0.602191	0.000000	0.0	0.000000	0.000000	0.000000	0.0

Figure 2: Tf-idf matrix.

7.1.4 Term frequency–inverse document frequency

Term frequency – Inverse document frequency (tf-idf) is a statistical algorithm that measures the features of all the items of a data set. This is achieved by counting the word occurrences in a text and linking them to the length of the text itself and the occurrence of the word in other documents.

- $tfidf(w, d) = tf(a, b) * idf(w, N)$
- $tf(w, d) = f(w, d) / \sum_k f(K, d)$
- $idf(w, N) = log(N/df(w))$
- $f(a, b)$: the frequency of the word a in the document b.
- $\sum_k f(K, d)$: The number of word in the document d.
- df_w: The number of documents containing the word w.
- N: The total amount of documents.

In our case this process has been used to quantify the genre features of all the movies and giving each movie a set of scores for each genre existent in the data set.

As it can be seen in the figures above, these are the results of tf-idf algorithm on the movie data set. The first row describes the feature scores of the first movie which can be classified mainly in the Children category (with a score of 0.518) but also in Fantasy (0.479) and Adventure (0.419).

7.1.5 Cosine similarity

In data science Cosine Similarity is a process of measuring the similarity between two items. The algorithm projects the items in a multidimensional space as

vectors and calculates the cosine of the angle between them. This being defined as the dot product of the vectors divided by the product of their lengths.

Since this process can only compare two items to one another and our algorithm has to work with multiple items as input we computed the mean of all the movies feature scores and created a single fictive movie to run cosine similarity on and compare it with all the others. The package Sklearn has been used to implement this algorithm. [7]

The figure below will explain the basic idea of cosine similarity in a 2 dimensional space.

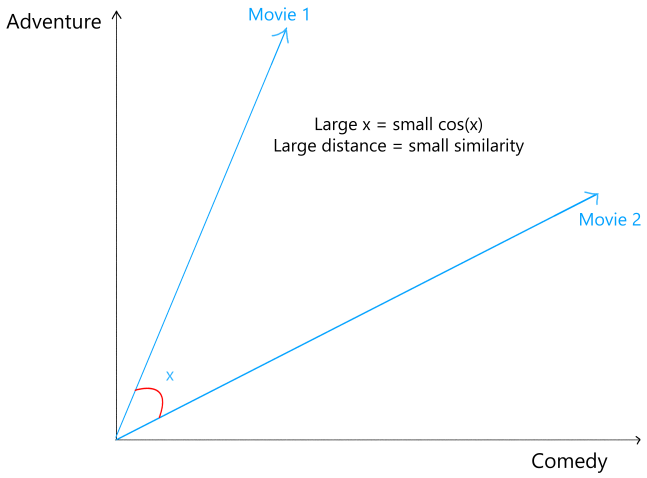


Figure 3: Cosine Similarity

As it can be seen the greater the angle distance between the points the smaller the cosine. This property is given by the fact that the cosine will always be inversely proportional and positive in the first quadrant.

Below we will go through the results and their interpretation:

	0	1	2	3	4	5	6	7	8	9	10
0	1.000000	0.813451	0.158804	0.143555	0.263711	0.000000	0.158804	0.656889	0.000000	0.266903	0.143555
1	0.813451	1.000000	0.000000	0.000000	0.000000	0.000000	0.000000	0.807534	0.000000	0.328112	0.000000
2	0.158804	0.000000	1.000000	0.903973	0.602191	0.000000	1.000000	0.000000	0.000000	0.000000	0.903973
3	0.143555	0.000000	0.903973	1.000000	0.544364	0.000000	0.903973	0.000000	0.000000	0.000000	1.000000
4	0.263711	0.000000	0.602191	0.544364	1.000000	0.000000	0.602191	0.000000	0.000000	0.000000	0.544364
5	0.000000	0.000000	0.000000	0.000000	0.000000	1.000000	0.000000	0.000000	0.576139	0.610693	0.000000
6	0.158804	0.000000	1.000000	0.903973	0.602191	0.000000	1.000000	0.000000	0.000000	0.000000	0.903973

Figure 4: Cosine similarity result matrix

After running Cosine similarity on the previous results provided by tf-idf we will be presented with similarity matrix which states the following: Each row

represents the genre similarity of a movie with all the movies in the data sets. As it can be seen the diagonal is filled with 1s because a movie will be identical to itself, thus the similarity will be 100%.

This matrix also contains our fictive movie which is a mean of all the genre scores of all the input movies. This row represents our results and can be used to rank the movies from most to least similar to our input.

7.1.6 User based collaborative filtering

User Based Collaborative Filtering is a data science technique of associating items with a score based on how much did similar users to our input liked the item that is being rated. This algorithm focuses on finding like minded people and sharing items from one to another.

7.1.7 User filtering

The process of filtering the users is relatively straightforward. The first step is selecting the users that have rated any of the movies our input has rated and grouping them by their user ID.

7.1.8 Pearson correlation coefficient

In data science, Pearson correlation coefficient (PCC) is an algorithm used to measure linear correlation between two sets of data. To be more precise, PCC computes the ratio between the product of the sets and the covariance of the two variables.

One of the main reasons this algorithm has been used for this approach is that Pearson correlation is not variant to scaling. This property is essential for our recommendation system because it focuses more on the in-between set item similarity rather than the rating of each item, preserving the similarity between user's movie decisions.

- $result = \frac{\sum_{i=1}^{N}(x_i - \overline{x_i})(y_i - \overline{y_i})}{\sqrt{\sum_{i=1}^{N}(x_i - \overline{x_i})^2 \sum_{i=1}^{N}(y_i - \overline{y_i})^2}}$
- x_i = x feature of the element.
- $\overline{x_i}$ = the mean of x feature of all the elements in the set.
- y_i = y feature of the element.
- $\overline{y_i}$ = the mean of x feature of all the elements in the set.
- N = the number of elements in the set.

The results given by the formula range from -1 to 1, where -1 represents a negative correlation and 1 represents a perfect correlation.

7.1.9 Computing the weights

After the user similarity is being computed using Pearson correlation, the last step in concluding the final score is quantifying the user similarity and the rating of each similar user for the movies to be recommended. Firstly, to obtain this score, a weighted rating is obtained by multiplying the user similarity by the user's rating on that movie. Secondly, after the weighted rating is obtained a sum is being applied to the top users' similarity and weighted rating after they are grouped by user ID. Finally, the movie recommendation score is obtained by dividing each movie's weighted rating sum by the similarity sum.

7.1.10 Multi objective Optimization

One obstacle we had to overcome while implementing this algorithm was the difficulty of predicting something so subjective as preference. To mitigate this problem we decided to not only show the best results but also rank them. By doing this we provide the user with the ability to browse through the recommendations and pick the items that they like the most. This process has been achieved by using multi criteria optimization.

7.1.11 Pareto ranking

Pareto ranking is a term used in multi objective optimization, an area of multiple criteria decision making that is comprised of optimizing multiple functions simultaneously.

Pareto set is represented by the list of all Pareto efficient options, which means that all the solutions inside the set cannot be dominated by any other solution.

Pareto ranking splits the data into multiple Pareto sets and treats them like shells. These shells help in organizing the data from best to worst in terms of criteria results. In our approach, the first shell contains the best options, whereas the last shell represents the worst options.

This approach has been used in multiple fields of science, such as engineering, logistics, and economics. Generally, this solution has been instrumented in cases where the problem presents more than one objective to be optimized or where there exist one or more trade-offs between multiple conflicting criteria.

In our scenario, the two functions we are trying to maximize are the algorithms mentioned above: Content based collaborative filtering and User based collaborative filtering. Each movie is associated with 2 scores and we are trying to find the movies with the biggest scores for both functions.

The figure below shows a scatter plot of the results and the Pareto front consisting of 3 elements. As it can be seen the red points represent the elements of the Pareto frontier.

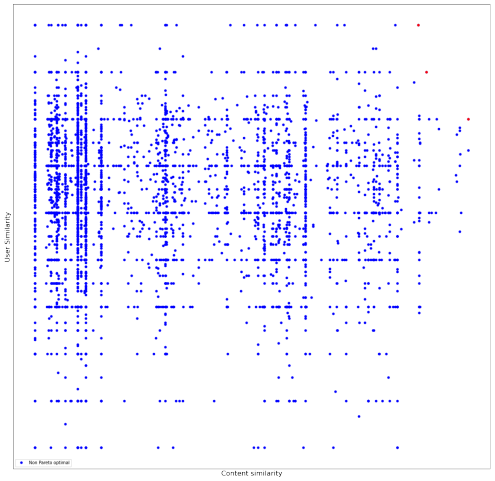


Figure 5: Ranking plotting

7.1.12 Deep neural network

As we decided on our most similar movies to our input by using the two filtering techniques and ordered them accordingly, our last step in offering recommendations is feeding these movie IDs into a deep neural network designed to predict the user rating of the movie based on all the ratings of all the current users from the data set. This neural network takes two arrays as input, one being the user id array which consists of all the user IDs in the data set, sometimes repeated since users rate multiple movies, and the second array consists of all the movies rated by the users, sometimes spanning over the data set multiple times since a movie will most likely be rated by multiple users. These inputs follow two different routes in this neural network both of them having the same algorithms applied to them. After both inputs enter the neural network an embedding is being run over them and the results are being flattened afterwards and joined together. Afterwards, both results are run through 2 dense layers to produce the final output of our system.

When compiling the model the most recurrent loss function and optimizer were used. For the loss, function mean squared error has been used and as an optimizer, we have used the Adam optimizer.

As it can be seen in the above figure, the deep learning model implemented consists of 8 different layers of which two are input layers, two are embedding layers, two are flattening layers, one is a concatenate, layer and finally, there are two dense layers including the final output layers which are being connected by a dropout layer.

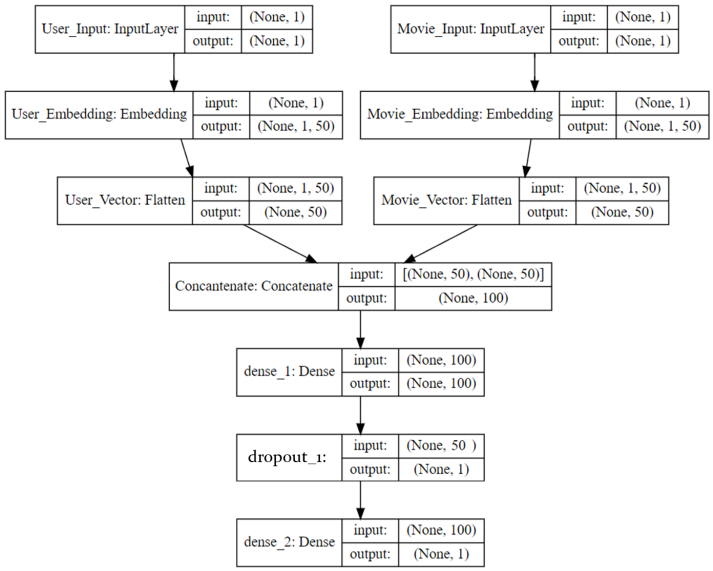


Figure 6: Model representation

7.1.13 Embedding layer

Embedding is a way of converting discrete objects into vectors of continuous values. These can be used to find similarities between items as they can be seen as points in a virtual space with different distances that can be used to determine the final similarity score.

7.1.14 Concatenate layer

A concatenate layer takes multiple inputs and merges them along a specified dimension. The inputs dimensions must be of the same size in all dimensions but the one to be concatenated.

7.1.15 Dense layer

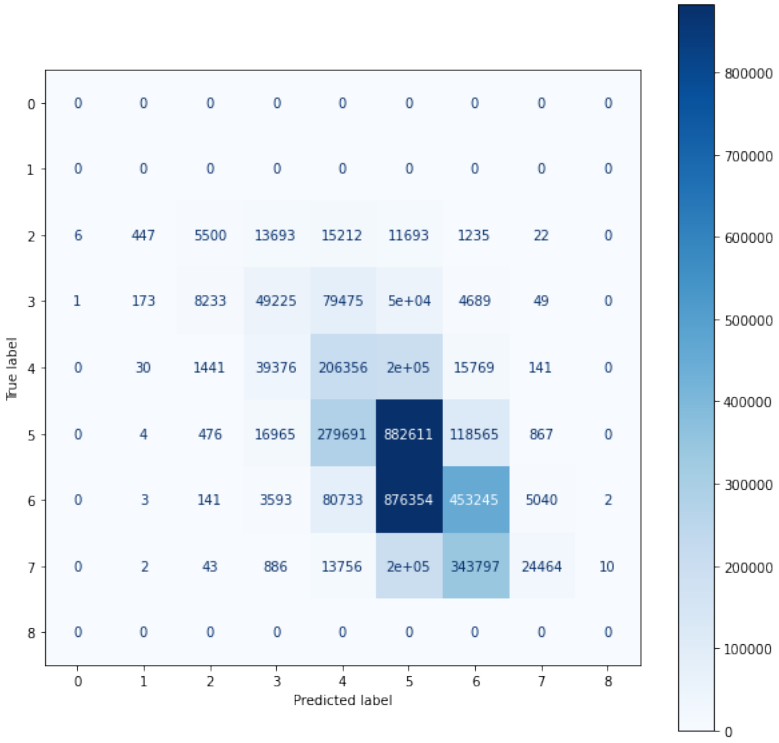
The dense layer is one of the most recurrent layers in neural networks these days. Dense layers are deeply connected which means that each neuron component in the dense layer receives input from all the neurons from the previous layer. These dense layers preform a matrix-vector multiplication. The parameters used in the matrix can be trained and updated with the use of back propagation. The dense layers created for this project use Relu activation as this is one of the most mainstream activation functions.

7.1.16 Model evaluation

To train and test the model we split the previously utilized Movie Lens user rating data set and split it to an 80 to 20 ratio, 80% representing the training test and 20% representing the testing set. The measures we used to evaluate the model results are root means squared evaluation (RMSE), accuracy, recall, and f1-score. The evaluation results of the previously mentioned metrics are the following:

- RMSE: 0.8073
- Accuracy: 48%
- Recall: 41%
- F1-score: 0.37%

To also get a better understanding of the computed predictions a confusion matrix has been computed and the result will be shown in the figure below:



7.2 Results

This recommender system consists of multiple methods working together towards analyzing the input and comparing it to the resources offered as well as labeling and sorting the items based on relevance. Initially the aim of this system was to offer you a singular movie as a recommendation that would have been chosen by the rating predictions of the neural network. But since the accuracy of the deep learning model was lacking, we decided to offer the user an array of movies with a relevance rating attached to it. Rating which is provided by the neural network. Below there have been attached 2 figures representing the input and the output.

Figure 7: Input

	movieId	title_year	genres	title	year	rating
0	456	Fresh (1994)	Crime Drama Thriller	Fresh	1994.0	5.0
1	1407	Scream (1996)	Comedy Horror Mystery Thriller	Scream	1996.0	4.5
2	6379	Wrong Turn (2003)	Horror Thriller	Wrong Turn	2003.0	5.0
3	7127	Run (1991)	Action Drama Thriller	Run	1991.0	4.9

Figure 8: Output

title	genres	year	prediction
House on Haunted Hill	Drama Horror Thriller	1959.0	2.433960
Final Conflict, The (a.k.a. Om...	Horror Thriller	1981.0	3.301464
Manhunter	Action Crime Drama Horror Thriller	1986.0	3.045309
Ginger Snaps	Drama Horror Thriller	2000.0	3.135657

7.3 Programming languages & Packages

As with most of the machine learning projects, this algorithm has been developed using python and Project Jupyter. Some of the notable packages used in this project are:

7.3.1 Tensorflow

Tensorflow is a package that offers a variety of structures used to develop and deploy models in python.

For more information please access this link:

7.3.2 Keras

Keras is a package that wraps Tensorflow and offers you a tidy workflow for developing deep learning models. This package was used for the development of the rating predictor neural network.

For more information please access this link:

7.3.3 Pandas

Pandas is a python package that provides structures required for loading visualizing and manipulating large amounts of data. This package was used to load

the data sets, create the user input and visualize the data as it is being worked on.

For more information please access this link:

7.3.4 Scikit-learn

Scikit-learn (Sklearn) is a very useful package for machine learning focused project. This package offers a variety of machine learning methods that work out of the box. this module has been used to compute Cosine similarity for Content based collaborative filtering.

For more information please access this link:

7.3.5 Numpy

Numpy is a package designed for array manipulation in python. This package provides a vast list of multidimensional structures (arrays, matrices) and methods that allow the application of mathematical operations over the arrays as well as shape manipulation. This package has been used to handle the data and feed it into the neural network.

For more information please access this link:

7.3.6 Matplotlib

Matplotlib is a python package that is mainly used to plot and visualize the data that needs to be processed. For this project Matplotlib has been used to visualize the data sets in order to get a better grasp on the trends of the features as well as verify the correctness of the results.

For more information please access this link:

7.3.7 Oapackage

This package has been used to implement the Pareto ranking algorithm on the final result. Since usually in optimization every problem has to be translated into a minimization task all the values from the final data matrix have been multiplied by -1 to match the algorithms criteria.

For more information please access this link:

7.3.8 Jupyter

Jupyter notebook was the used environment for this project hence it offers a large array of features such as: interface flexibility, interactivity, and last but not least plenty of configurable aspects that make any data science project a lot easier to handle.

For more information please access this link:

8 Project management

8.1 Timeline reflection

This project has represented a very interesting journey in itself. Looking back at the initial time scheduling and how the project took place I can confidently say that gathering data was a bigger challenge than expected. This underestimation has deferred my progress drastically. This led me to rush the implementation process of the algorithms. Another problem I came across is managing to test the whole algorithm, as for the deterministic filtering techniques the testing I came across did not prove any efficiency. Fortunately, the neural network's accuracy has been evaluated rigorously.

8.2 Change in direction

The initial aim of this project was to research different algorithms used for recommending items. As time went by the aim shifted to developing a rather new approach which combines 2 different filtering techniques respectively Content based collaborative filtering and User based collaborative filtering tying them together with Pareto ranking and predicting ratings on the first Pareto shell using a deep neural network.

8.3 Encountered difficulties & their mitigation

When embarking upon this journey I had high hopes and set myself some clear steps to follow in order to achieve the desired result. Down the way, I have encountered some difficulties, which together with my lack of experience in this field, – since this has been my first project on the topic of machine learning – have held me back and left me behind on the previously set time scheduling. Even though there were times when I felt aimless and hopeless I managed to power through and adapt to the changes in order to provide a good project.

The first obstacle faced in this project was gathering data sets. Even though the internet provides a wide variety of movie data sets, most of them would not offer exactly what I needed for this project. Therefore, I decided to go with MovieLens mainly because it provides user rates history in a very satisfying format.

One other obstacle I met with in this project was labeling preference for new input. This problem has been mitigated by using deterministic approaches that create scores based on similarity and by offering multiple options, which I achieved by using multi-objective optimization – namely Pareto Ranking.

9 Conclusion

This project represented an interesting journey of discovery and learning about new things. Furthermore, it marks my first machine learning personal project. I am extremely proud to state that the approach I used ended up providing

plausible results and also, managed to diminish the cold start problem. Even though there were no models involved, this algorithm offers a solid start for a larger recommendation system pipeline as the results concluded can be fed into other machine learning models for predictions. I am also glad that I had the opportunity to involve an optimization algorithm, since I find that machine learning and optimization are two science fields that go well together.

9.1 Lessons learned

This project was first of all a learning process which taught me a myriad of lesson – from gathering data, to comparing approaches and evaluating advantages and drawbacks and to implementing machine learning techniques and models.

The main thing I will take away from this project is that learning about a new field or topic is a thorough process in itself, especially with machine learning, a computer science field which abounds of terms and approaches. Furthermore, another clear take away is that when working with large data there is a lot of noise that has to be spotted and mitigated. At the end of the day, models do not understand the data they work with which means that the logic behind the models has to be as precise as possible.

Lastly, a conclusion that was drawn from this project is that the evaluation of the models offers a clear view on not only the performance of the model but also the flaws of the execution of the model itself if any. Evaluation is a key process that should never be neglected even though sometimes can be more difficult than the development of the product itself.

9.2 Future work

The current algorithm manages to generate some trivial recommendations due to the rudimentary nature of the data sets. This is left to be worked on in the future as there are plenty of ways that could lead to improvement.

The first step in improving the addressed recommendation system is training the deep learning model more and increasing its precision and accuracy. Another point which would lead to better results is upgrading the multi-objective sorting from a 2d sort to a multi dimensional sorting that also includes IMDB ratings as well as movie revenue and the number of votes.

Another optimization that can lead the movie recommendation system to better suggestions is the improvement of the deep learning model currently utilized to show a predicted rating. If the model becomes more accurate, its predictions can be used as a final threshold that the movies have to pass to make it to the final recommendation.

Finally, the introduction of features to the algorithm will bring more information to be computed and also will provide the recommendation system with a clearer view of the task. The first step in doing this has been executed, since the project notebook also provides a section which focuses on combining the IMDB data set with MovieLens to not only provide a rating history but also feature-rich movie descriptions.

9.3 Summary

All being said, I can gladly say that this project has created a functional system that provides somewhat accurate recommendations. The making process has taught me a lot of new concepts regarding computer science such as proper techniques for gathering and cleaning a data set, approaches to filtering items based on relevance (Content-filtering Memory-filtering), multi criteria optimization (Pareto Ranking) and the use of deep learning models in recommendation systems.

This project has also led me to understand how big companies, such as Netflix, Spotify and YouTube tackle the specified problem and what tools they use.

Data Science is an ever-expanding field that provides solutions to a variety of tasks and problems. This project has attempted to offer a lightweight solution to movie recommendation and propose an algorithm which combines Collaborative Filtering with Deep Neural Networks to generate suggestions.

I would like to once again thank everybody that has supported me during this project and also thank the reader for their attention.

References

[1] Manzar Abbas et al. "Context-aware Youtube recommender system". In: *2017 international conference on information and communication technologies (ICICT)*. IEEE. 2017, pp. 161–164.

[2] Xavier Amatriain. "Big & personal: data and models behind netflix recommendations". In: *Proceedings of the 2nd international workshop on big data, streams and heterogeneous source Mining: Algorithms, systems, programming models and applications*. 2013, pp. 1–6.

[3] Ashton Anderson et al. "Algorithmic effects on the diversity of consumption on spotify". In: *Proceedings of The Web Conference 2020*. 2020, pp. 2155–2165.

[4] Carlos A Gomez-Uribe and Neil Hunt. "The netflix recommender system: Algorithms, business value, and innovation". In: *ACM Transactions on Management Information Systems (TMIS)* 6.4 (2015), pp. 1–19.

[5] Jonathan L Herlocker, Joseph A Konstan, and John Riedl. "Explaining collaborative filtering recommendations". In: *Proceedings of the 2000 ACM conference on Computer supported cooperative work*. 2000, pp. 241–250.

[6] *Spotify AB*. `https://newsroom.spotify.com/company-info/`. Accessed: 2021-10-27.

[7] Wikipedia contributors. *Cosine similarity — Wikipedia, The Free Encyclopedia*. [Online; accessed 19-April-2022]. 2022. URL: `https://en.wikipedia.org/w/index.php?title=Cosine_similarity&oldid=1071254454`.

10 Appendix

The implementation of the system has been inspired by the following two sources:

Source 1

Source 2

Below there have been attached screenshots of the project's notebooks.

```
from __future__ import print_function
import pandas as pd
import scipy
from scipy.sparse import coo_matrix, vstack
import math
import numpy as np
import matplotlib.pyplot as plt
import pygmo as pg
import oapackage
from sklearn.metrics.pairwise import linear_kernel
from sklearn.feature_extraction.text import TfidfVectorizer
from sklearn.decomposition import FactorAnalysis
%matplotlib inline
```

Import datasets

```
# Movie-lens
movies = pd.read_csv('E:\\University Stuff\\Third year\\movie_recommender_research\\datasets\\movie_lens_2m\\movies.csv')
links = pd.read_csv('E:\\University Stuff\\Third year\\movie_recommender_research\\datasets\\movie_lens_2m\\links.csv')
```

Clean the data

```
# the function to extract titles
def extract_title(title):
    year = title[len(title)-5:len(title)-1]

    # some movies do not have the info about year in the column title. So, we should take care of the case as well.

    if year.isnumeric():
        title_no_year = title[:len(title)-7]
        return title_no_year
    else:
        return title

# the function to extract years
def extract_year(title):
    year = title[len(title)-5:len(title)-1]
    # some movies do not have the info about year in the column title. So, we should take care of the case as well.
    if year.isnumeric():
        return int(year)
    else:
        return np.nan
```

```
movies.rename(columns={'title':'title_year'}, inplace=True)
# remove whitespaces in title_year
movies['title_year'] = movies['title_year'].apply(lambda x: x.strip())
# create the columns for title and year
movies['title'] = movies['title_year'].apply(extract_title)
movies['year'] = movies['title_year'].apply(extract_year)
```

Remove movies with no genres

```
print('Amount of entries before null genre removal:',len(movies))
movies = movies[~(movies['genres']=='(no genres listed)')].reset_index(drop=True)
print('Amount of entries after null genre removal:',len(movies))
```

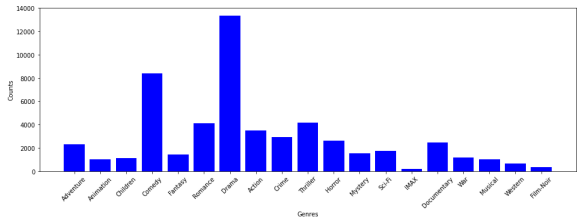
```
Amount of entries before null genre removal: 27278
Amount of entries after null genre removal: 27032
```

Counting the number of occurrences for each

```
# remove '|' in the genres column
movies['genres'] = movies['genres'].str.replace('|',' ')


# Populate the dictionary
counts = {}
for i in movies.index:
    for g in movies.loc[i, 'genres'].split(' '):
        if g not in counts:
            counts[g] = 1
        else:
            counts[g] = counts[g] + 1
```

```
# create a bar chart
f,ax = plt.subplots(1,1,figsize=(16,5))
plt.bar(list(counts.keys()), counts.values(), color='b')
plt.xticks(rotation=45)
plt.xlabel('Genres')
plt.ylabel('Counts')
```

```
Text(0, 0.5, 'Counts')
```



Creating User Input

```
movies.head(1)
```

	movieId	title_year	genres	title	year
0	1	Toy Story (1995)	Adventure Animation Children C...	Toy Story	1995.0

1 rows × 5 columns Open in new tab

```
user = [
           {'title':'Run', 'rating':5.0},
           {'title':'Fresh', 'rating':5.0},
           {'title':'Wrong Turn', 'rating':5.0},
           {'title':"Old", 'rating':5.0},
           {'title':'Scream', 'rating':4.5}
        ]
# Prevent Pearson correlation bug
if user[0]['rating'] >= 1:
    user[0]['rating'] -= 0.1
input_movie = pd.DataFrame(user)
#Filtering by title
Id = movies[movies['title'].isin(input_movie['title'].tolist())]
#merging it by title.
input_movie = pd.merge(Id, input_movie)
print(movies.shape)
```

```
input_movie
```

	movieId	title_year	genres	title	year	rating
0	456	Fresh (1994)	Crime Drama Thriller	Fresh	1994.0	5.0
1	1407	Scream (1996)	Comedy Horror Mystery Thriller	Scream	1996.0	4.5
2	6379	Wrong Turn (2003)	Horror Thriller	Wrong Turn	2003.0	5.0
3	7127	Run (1991)	Action Drama Thriller	Run	1991.0	4.9

Content based collaborative filtering

Source

Implementations Steps

- Step 1: Quantify features using tf-idf
- Step 2: Calculate similarity

Term Frequency and Inverse Document Frequency (tf-idf)

tf-idf is a numerical statistic which is used to calculate the importance of a word to a document in a collection of documents.

tf-idf (i, j) = tf (i, j) × idf (i, N)

```
# swap 'Sci-Fi' to 'SciFi' and 'Film-Noir' to 'Noir'
movies['genres'] = movies['genres'].str.replace('Sci-Fi','SciFi')
movies['genres'] = movies['genres'].str.replace('Film-Noir','Noir')

# create TfidfVectorizer
tfidf_vector = TfidfVectorizer(stop_words='english')

# apply the object to the genres column
tfidf_matrix = tfidf_vector.fit_transform(movies['genres'])
print(tfidf_matrix.shape)
```

```
tfidf_df = pd.DataFrame.sparse.from_spmatrix(tfidf_matrix)
```

Question?

```
Would it be effective to?
```

1. Create a fictive movie that is a mean of all the generes of the input movies
2. introduce it in the matrix above
3. run cosine similarity with all the other movies
4. get the recommendations based on the similarity with the fictive movie

```
def create_movie_mean(input_movie):
    temp_df = pd.DataFrame(columns=range(19))
    for m_id in input_movie['movieId']:
        i = int(movies.loc[movies['movieId'] == m_id].index[0])
        temp_df = temp_df.append(pd.DataFrame.sparse.from_spmatrix(tfidf_matrix.getrow(i)))
    return temp_df.mean()

movie_mean_sparse = scipy.sparse.csr_matrix(create_movie_mean(input_movie))
tfidf_matrix = vstack([tfidf_matrix, movie_mean_sparse])
```

- The tfidf_matrix is the matrix with 27032 rows(movies) and 19 columns(genre).
- The row number of the matrix corresponds to the index of the movies data frame

```
# Calculate sum of features
row = tfidf_matrix.getrow(0)
row = pd.DataFrame.sparse.from_spmatrix(row)
tfidf_score = 0
for i in row.iteritems():
    for j in i[1].iteritems():
        tfidf_score += j[1]
print(tfidf_score)
```

```
2.18683413255787
```

```
print(list(enumerate(tfidf_vector.get_feature_names_out())))
```

```
[(0, 'action'), (1, 'adventure'), (2, 'animation'), (3, 'children'), (4, 'comedy'), (5, 'crime'), (6,
'documentary'), (7, 'drama'), (8, 'fantasy'), (9, 'horror'), (10, 'imax'), (11, 'musical'), (12, 'mystery'),
(13, 'noir'), (14, 'romance'), (15, 'scifi'), (16, 'thriller'), (17, 'war'), (18, 'western')]
```

Toy Story: Adventure(1 = 0.4190593152859304)
Animation(2 = 0.5184153567262332)
Children(3 = 0.5058584647541116)
Comedy(4 = 0.2637107911959773)
Fantasy(8 = 0.4797902045956178)

Since “Animation value” is the biggest one this means that Toy story is most likely an animation

```
tfidf_df
```

```
tfidf_df
```

	0	1	2	3	4	5	6	7	8	9
0	0.0	0.419059	0.518415	0.505858	0.263711	0.0	0.0	0.000000	0.479790	0.0
1	0.0	0.515162	0.000000	0.621867	0.000000	0.0	0.0	0.000000	0.589821	0.0
2	0.0	0.000000	0.000000	0.000000	0.602191	0.0	0.0	0.000000	0.000000	0.0
3	0.0	0.000000	0.000000	0.000000	0.544364	0.0	0.0	0.427589	0.000000	0.0
4	0.0	0.000000	0.000000	0.000000	1.000000	0.0	0.0	0.000000	0.000000	0.0
...	...	...	...	...	...	...	...	...	...	...
27027	0.0	0.000000	0.000000	0.000000	0.545482	0.0	0.0	0.000000	0.000000	0.8
27028	0.0	0.000000	0.000000	0.000000	1.000000	0.0	0.0	0.000000	0.000000	0.0

```
# create the cosine similarity matrix
sim_matrix = linear_kernel(tfidf_matrix,tfidf_matrix)
```

```
sim_matrix
```

	0	1	2	3	4	5	6	7	8
0	1.000000	0.813451	0.158804	0.143555	0.263711	0.000000	0.158804	0.656889	0.00000
1	0.813451	1.000000	0.000000	0.000000	0.000000	0.000000	0.000000	0.807534	0.00000
2	0.158804	0.000000	1.000000	0.903973	0.602191	0.000000	1.000000	0.000000	0.00000
3	0.143555	0.000000	0.903973	1.000000	0.544364	0.000000	0.903973	0.000000	0.00000
4	0.263711	0.000000	0.602191	0.544364	1.000000	0.000000	0.602191	0.000000	0.00000
5	0.000000	0.000000	0.000000	0.000000	0.000000	1.000000	0.000000	0.000000	0.57613
6	0.158804	0.000000	1.000000	0.903973	0.602191	0.000000	1.000000	0.000000	0.00000
7	0.656889	0.807534	0.000000	0.000000	0.000000	0.000000	0.000000	1.000000	0.00000

27033 rows × 501 columns Open in new tab

```
def get_id_from_title(title):
    return movies[movies.title == title]['movieId'].values[0]

def calculate_ratings(title):
    return get_id_from_title(title)
```

```
def contents_based_recommender():
    movie_index = sim_matrix.shape[0] - 1
    movie_list = list(enumerate(sim_matrix[int(movie_index)]))
    similar_movies =  pd.DataFrame(list(filter(lambda x:x[0] != int(movie_index), sorted(movie_list,key=lambda x:x[1], reverse=True)
     )), columns=['movieId', 'Similarity'])
    return similar_movies
```

```
similarity_result = contents_based_recommender()

```

```
similarity_result
```

	movieId	Similarity
0	90	0.654889
1	1054	0.654889
2	1308	0.654889
3	1598	0.654889
4	2076	0.654889
5	2403	0.654889
6	2434	0.654889
7	2466	0.654889

User based collaborative filtering

source

```
ratings = pd.read_csv('E:\\University Stuff\\Third year\\movie_recommender_research\\datasets\\movie_lens_2m\\ratings.csv')
movie = pd.read_csv('E:\\University Stuff\\Third year\\movie_recommender_research\\datasets\\movie_lens_2m\\movies.csv')
```

```

movie['year'] = movie.title.str.extract('(\\d\d\d\d\))',expand=False)
#Removing the parentheses
movie['year'] = movie.year.str.extract('(\d\d\d\d)', expand=False)
#Removing the years from the 'title' column
movie['title'] = movie.title.str.replace('(\(\d\d\d\d\))', '')
#Applying the strip function to get rid of any ending whitespace characters that may have appeared
movie['title'] = movie['title'].apply(lambda x: x.strip())
movie.drop(columns=['genres'], inplace=True)
ratings.drop(columns=['timestamp'], inplace=True)

```

```
def filter_users(given_ratings):
    #Filtering out users that have watched movies that the input has watched and storing it
    users = given_ratings[ratings['movieId'].isin(input_movie['movieId'].tolist())]
    user_subset_group = users.groupby(['userId'])
    #Sorting it so that users with movie ratings most in common with the input will have priority
    user_subset_group = sorted(user_subset_group, key=lambda x: len(x[1]), reverse=True)
    return user_subset_group
```

```
user_subset_group = filter_users(ratings)
```

```
user_subset_group = filter_users(ratings)
```

```
#Store the Pearson Correlation in a dictionary, where the key is the user Id and the value is the coefficient
pearsonCorDict = {}

#For every user group in our subset
for name, group in user_subset_group:
    #Let's start by sorting the input and current user group so the values aren't mixed up later on
    group = group.sort_values(by='movieId')
    input_movie = input_movie.sort_values(by='movieId')
    #Get the N for the formula
    n = len(group)
    #Get the review scores for the movies that they both have in common
    temp = input_movie[input_movie['movieId'].isin(group['movieId'].tolist())]
    #And then store them in a temporary buffer variable in a list format to facilitate future calculations
    tempRatingList = temp['rating'].tolist()
    #put the current user group reviews in a list format
    tempGroupList = group['rating'].tolist()
    #Now let's calculate the pearson correlation between two users, so called, x and y
    Sxx = sum([i**2 for i in tempRatingList]) - pow(sum(tempRatingList),2)/float(n)
    Syy = sum([i**2 for i in tempGroupList]) - pow(sum(tempGroupList),2)/float(n)
    Sxy = sum( i*j for i, j in zip(tempRatingList, tempGroupList)) - sum(tempRatingList)*sum(tempGroupList)/float(n)

    #If the denominator is different than zero, then divide, else, 0 correlation.
    if Sxx != 0 and Syy != 0:
        pearsonCorDict[name] = Sxy/math.sqrt(Sxx*Syy)
    else:
        pearsonCorDict[name] = 0
```

```
pearson_df = pd.DataFrame.from_dict(pearsonCorDict, orient='index')
pearson_df.columns = ['similarityIndex']
pearson_df['userId'] = pearson_df.index
pearson_df.index = range(len(pearson_df))
pearson_df.head()
```

	similarityIndex	userId
0	-0.924658	59477
1	-0.619930	127548
2	-1.000000	4450
3	0.300376	6976
4	-0.654654	8405

5 rows × 2 columns Open in new tab

```
topUsers= pearson_df.sort_values(by='similarityIndex', ascending=False)[0:50]
topUsers.head()
```

	similarityIndex	userId
322	1.0	39556
203	1.0	22163
658	1.0	96788
352	1.0	46470
878	1.0	134175

```
topUsersRating = topUsers.merge(ratings, left_on='userId', right_on='userId', how='inner')
topUsersRating.head(1)
```

	similarityIndex	userId	movieId	rating
0	1.0	39556	1	4.0

1 rows × 4 columns Open in new tab

```
#Multiplies similarity by user ratings
topUsersRating['weightedRating'] = topUsersRating['similarityIndex'] * topUsersRating['rating']
topUsersRating.head(1)
```

	similarityIndex	userId	movieId	rating	weightedRating
0	1.0	39556	1	4.0	4.0

1 rows × 5 columns Open in new tab

```
#Applies a sum to the topUsers after grouping it up by userId
tempTopUsersRating = topUsersRating.groupby('movieId').sum()[['similarityIndex', 'weightedRating']]
tempTopUsersRating.columns = ['sum_similarityIndex', 'sum_weightedRating']
tempTopUsersRating.head(1)
```

movieId	sum_similarityIndex	sum_weightedRating
1	42.0	152.0

1 rows × 2 columns Open in new tab

```
recommendation_df = pd.DataFrame()
#take the weighted average
recommendation_df['weighted average recommendation score'] = tempTopUsersRating['sum_weightedRating'] /
 tempTopUsersRating['sum_similarityIndex']
recommendation_df['movieId'] = tempTopUsersRating.index
recommendation_df.head(1)
```

movieId	weighted average recommendation	movieId
1	3.619048	1

1 rows × 2 columns Open in new tab

```
recommendation_df = recommendation_df.sort_values(by='weighted average recommendation score', ascending=False)

```

```
movie.loc[movie['movieId'].isin(recommendation_df['movieId'].tolist())]
```

	movieId	title	year
0	1	Toy Story	1995
1	2	Jumanji	1995
2	3	Grumpier Old Men	1995
3	4	Waiting to Exhale	1995
4	5	Father of the Bride Part II	1995
5	6	Heat	1995
6	7	Sabrina	1995
7	8	Tom and Huck	1995

```
recommendation_df.reset_index(drop = True, inplace = True)
```

```
collaborative_final = recommendation_df.merge(similarity_result, how='inner', on='movieId')
```

```
collaborative_final_ids = collaborative_final['movieId']
collaborative_final = np.asarray(collaborative_final[['Similarity', 'weighted average recommendation score']])
```

Pareto ranking

This part of the project focuses on implementing the pareto ranking algorithm on the 2d dataset containing the previous filtering results.

```
pareto=oapackage.ParetoDoubleLong()
for ii in range(0, collaborative_final.shape[1]):
    w=oapackage.doubleVector( (collaborative_final[ii,0], collaborative_final[ii,1]))
    pareto.addvalue(w, ii)
pareto.show(verbose=2)
```

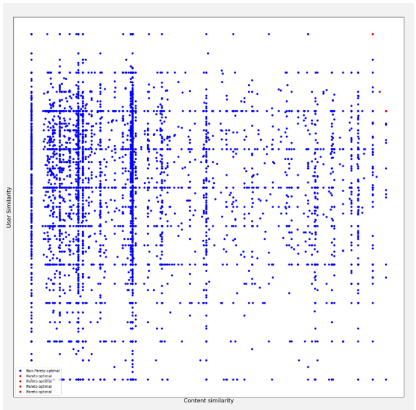
```
 Pareto: 1 optimal values, 1 objects
 value 0: 0.187414; 5.000000
```

```
lst=pareto.allindices() # the indices of the Pareto optimal designs
optimal_datapoints=collaborative_final[:,lst]
```

```
ndf, dl, dc, ndr = pg.fast_non_dominated_sorting(points = -collaborative_final[:,:])
```

```
f = plt.figure()
f.set_figwidth(20)
f.set_figheight(20)

h=plt.plot(collaborative_final[:,0], collaborative_final[:,1], '.b', markersize=10, label='Non Pareto optimal')
for i in ndf[0]: # select pareto shell 0 = best
    print('Stats:',collaborative_final[i,:])
    hp=plt.plot(collaborative_final[i,0], collaborative_final[i,1], '.r', markersize=10, label = 'Pareto optimal')
plt.xlabel('Content similarity', fontsize=16)
plt.ylabel('User Similarity', fontsize=16)
plt.xticks([])
plt.yticks([])
_=plt.legend(loc=3, numpoints=1)

```

```
Stats: [0.63007737 5.        ]
Stats: [0.6430501 4.25     ]
Stats: [0.65488938 4.        ]
Stats: [0.65488938 4.        ]
```




```
movie_ids = []
for i in ndf[0]:
    movie_ids.append(int(movies.iloc[collaborative_final_ids[i]][['movieId']]))
    print(int(movies.iloc[collaborative_final_ids[i]][['movieId']]))
result = movies.loc[movies['movieId'].isin(movie_ids)]

```

```
2790
4210
2519
6820
```

DNN recommendations

```
ratings = pd.read_csv('E:\\University Stuff\\Third year\\movie_recommender_research\\datasets\\movie_lens_2m\\ratings.csv')
```

```
from keras.models import load_model
model = load_model('E:\\University Stuff\\Third year\\movie_recommender_research\\notebooks\\Models\\Model notebook')
```

```
next_user = ratings['userId'].max()
dnn_input = pd.DataFrame(columns=['userId', 'movieId'])
dnn_input['movieId'] = movie_ids
dnn_input['userId'] = next_user
```

```
final_pred = model.predict([dnn_input['userId'], dnn_input['movieId']])
```

```
result['prediction'] = final_pred
```

```
output = result.loc[result['movieId'] == collaborative_final_ids.iloc[final_pred.argmax()]]
```

```
collaborative_final_ids.iloc[1]
```

```
4301
```

```
result[['title', 'genres', 'year', 'prediction']].head()
```

	title	genres	year	prediction
2434	House on Haunted Hill	Drama Horror Thriller	1959.0	2.433960
2704	Final Conflict, The (a.k.a. Om...	Horror Thriller	1981.0	3.301464
4116	Manhunter	Action Crime Drama Horror Thri...	1986.0	3.045309
6710	Ginger Snaps	Drama Horror Thriller	2000.0	3.135657

```
import pandas as pd
import numpy as np
import matplotlib.pyplot as plt
from keras.layers import Dense, Dropout, Flatten, Input
from keras.layers import Conv2D, MaxPooling2D, BatchNormalization
from keras.layers import Embedding, Input, dot, concatenate
from keras.models import Model
from sklearn.model_selection import train_test_split
```

DNN Movie rating predictor

```
ratings = pd.read_csv('E:\\University Stuff\\Third
 year\\movie_recommender_research\\datasets\\movie_lens_2m\\ratings.csv')
movie = pd.read_csv('E:\\University Stuff\\Third
 year\\movie_recommender_research\\datasets\\movie_lens_2m\\movies.csv')
```

```
X = ratings.iloc[:,:2]
Y = ratings.iloc[:,2]

x_train, x_test, y_train, y_test = train_test_split(X, Y, test_size = 0.2, random_state = 66)
```

```
# The number of latent factors for the embedding
n_latent_factors = 30

# no of users and movies
n_users, n_movies = len(ratings['userId'].unique()), len(ratings['movieId'].unique())
```

```
# Model Architecture


# User Embeddings
user_input = Input(shape=(1,), name='User_Input')
user_embeddings = Embedding(input_dim = n_users, output_dim=n_latent_factors, input_length=1,
                            name='User_Embedding') (user_input)
user_vector = Flatten(name='User_Vector') (user_embeddings)



# Movie Embeddings
movie_input = Input(shape=(1,), name='Movie_Input')
movie_embeddings = Embedding(input_dim = n_movies, output_dim=n_latent_factors, input_length=1,
                             name='Movie_Embedding') (movie_input)
movie_vector = Flatten(name='Movie_Vector') (movie_embeddings)

# Concatenate Product
merged_vectors = concatenate([user_vector, movie_vector], name='Concantenate')
first_dense = Dense(50, activation='relu')(merged_vectors)
dense_layer_1 = Dropout(0.25) (first_dense)
batchnorm = BatchNormalization()(dense_layer_1)
dense_layer_2 = Dense(32, activation='relu')(merged_vectors)


result = Dense(1)(dense_layer_1)
model = Model([user_input, movie_input], result)
```

```
model.summary()
```

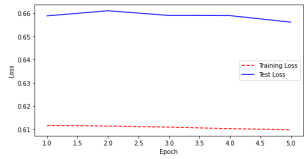
```
Model: "model_1"
__________________________________________________________________________________________________
 Layer (type)                   Output Shape         Param #     Connected to
==================================================================================================
 User_Input (InputLayer)        [(None, 1)]          0           []

 Movie_Input (InputLayer)       [(None, 1)]          0           []

 User_Embedding (Embedding)     (None, 1, 15)        2077395     ['User_Input[0][0]']

 Movie_Embedding (Embedding)    (None, 1, 15)        401160      ['Movie_Input[0][0]']
```

```
model.compile(loss='mean_squared_error', optimizer = "Adam")
```

```
batch_size = 128
epochs = 5
history = model.fit(x=[x_train['userId'], x_train['movieId']], y=y_train, batch_size= batch_size,
 epochs=epochs,
                    verbose= 1, validation_data=([x_test['userId'], x_test['movieId']], y_test))
```

```
Epoch 1/5
125002/125002 [==============================] - 943s 8ms/step - loss: 0.6117 - val_loss: 0.6589
Epoch 2/5
125002/125002 [==============================] - 993s 8ms/step - loss: 0.6114 - val_loss: 0.6611
Epoch 3/5
125002/125002 [==============================] - 1244s 10ms/step - loss: 0.6110 - val_loss: 0.6590
```

```python
# Get training and test loss histories
training_loss = history.history['loss']
test_loss = history.history['val_loss']

# Create count of the number of epochs
epoch_count = range(1, len(training_loss) + 1)

# Visualize loss history
plt.figure(figsize = (8,4))
plt.plot(epoch_count, training_loss, 'r--')
plt.plot(epoch_count, test_loss, 'b-')
plt.legend(['Training Loss', 'Test Loss'])
plt.xlabel('Epoch')
plt.ylabel('Loss')
plt.show()
```




```python
score = model.evaluate([x_test['userId'], x_test['movieId']], y_test)

print()
print('RMSE: {:.4f}'.format(np.sqrt(score)))
```

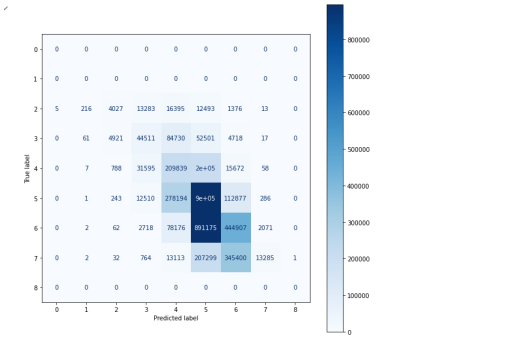
```
125002/125002 [==============================] - 320s 3ms/step - loss: 0.6561

RMSE: 0.8100
```

```python
predictions = model.predict(x = [x_test['userId'], x_test['movieId']])
```

```python
from sklearn.metrics import ConfusionMatrixDisplay, classification_report, confusion_matrix
conf_m = confusion_matrix(y_true= np.floor(y_test), y_pred= np.floor(predictions))
cmd = ConfusionMatrixDisplay(conf_m)
fig, ax = plt.subplots(figsize=(10,10))
cmd.plot(ax=ax, cmap=plt.cm.Blues)
```

```
<sklearn.metrics._plot.confusion_matrix.ConfusionMatrixDisplay at 0x16e318c6048>
```


<sklearn.metrics._plot.confusion_matrix.ConfusionMatrixDisplay at 0x16e318c6048>



```
print(classification_report(y_true=np.floor(y_test), y_pred=np.floor(predictions)), 'zero_division')
```

```
E:\University Stuff\Third year\movie_recommender_research\venv\lib\site-packages\sklearn\metrics
\_classification.py:1318: UndefinedMetricWarning: Recall and F-score are ill-defined and being set
to 0.0 in labels with no true samples. Use `zero_division` parameter to control this behavior.
  _warn_prf(average, modifier, msg_start, len(result))
E:\University Stuff\Third year\movie_recommender_research\venv\lib\site-packages\sklearn\metrics
\_classification.py:1318: UndefinedMetricWarning: Recall and F-score are ill-defined and being set
to 0.0 in labels with no true samples. Use `zero_division` parameter to control this behavior.
  _warn_prf(average, modifier, msg_start, len(result))
```

	precision	recall	f1-score	support
-2.0	0.00	0.00	0.00	0
-1.0	0.00	0.00	0.00	0
0.0	0.40	0.08	0.14	47808
1.0	0.42	0.23	0.30	191459
2.0	0.31	0.45	0.37	462600
3.0	0.40	0.69	0.50	1299179
4.0	0.48	0.31	0.38	1419111
5.0	0.84	0.02	0.04	579896
6.0	0.00	0.00	0.00	0

```
E:\University Stuff\Third year\movie_recommender_research\venv\lib\site-packages\sklearn\metrics
\_classification.py:1318: UndefinedMetricWarning: Recall and F-score are ill-defined and being set
to 0.0 in labels with no true samples. Use `zero_division` parameter to control this behavior.
  _warn_prf(average, modifier, msg_start, len(result))
```